

# FPL Assistant: Graph-RAG System

Milestone 3 - ACL 2025

---

Team ACL\_M3

December 16, 2025

German University in Cairo

# Outline

1. High-Level System Architecture
2. Input Preprocessing
3. Graph Retrieval Layer - Baseline
4. Graph Retrieval Layer - Embeddings  
    Embedding-Based Retrieval
- LLM Layer
6. Error Analysis & Improvements
7. Live Demo

# **1. High-Level System Architecture**

---

# System Architecture

## Pipeline Overview

1. **Input:** User Query (Streamlit UI)

## Task & Dataset

# System Architecture

## Pipeline Overview

1. **Input:** User Query (Streamlit UI)
2. **Preprocessing:** Intent Class. + Entity Ext.

## Task & Dataset

# System Architecture

## Pipeline Overview

1. **Input:** User Query (Streamlit UI)
2. **Preprocessing:** Intent Class. + Entity Ext.
3. **Retrieval:** Hybrid (Cypher + Vector)

## Task & Dataset

# System Architecture

## Pipeline Overview

1. **Input:** User Query (Streamlit UI)
2. **Preprocessing:** Intent Class. + Entity Ext.
3. **Retrieval:** Hybrid (Cypher + Vector)
4. **LLM:** Context Augmentation + Generation

## Task & Dataset

# System Architecture

## Pipeline Overview

1. **Input:** User Query (Streamlit UI)
2. **Preprocessing:** Intent Class. + Entity Ext.
3. **Retrieval:** Hybrid (Cypher + Vector)
4. **LLM:** Context Augmentation + Generation

## Task & Dataset

- **Task:** FPL Expert (Stats, Analysis, Recommendations).

# System Architecture

## Pipeline Overview

1. **Input:** User Query (Streamlit UI)
2. **Preprocessing:** Intent Class. + Entity Ext.
3. **Retrieval:** Hybrid (Cypher + Vector)
4. **LLM:** Context Augmentation + Generation

## Task & Dataset

- **Task:** FPL Expert (Stats, Analysis, Recommendations).
- **Data:** Merged 2021-2023 Seasons (Cleaned).

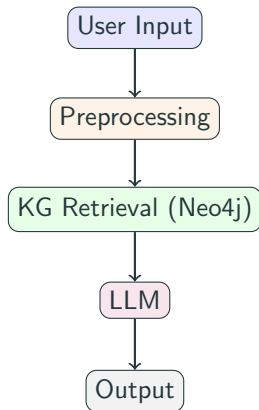
# System Architecture

## Pipeline Overview

1. **Input:** User Query (Streamlit UI)
2. **Preprocessing:** Intent Class. + Entity Ext.
3. **Retrieval:** Hybrid (Cypher + Vector)
4. **LLM:** Context Augmentation + Generation

## Task & Dataset

- **Task:** FPL Expert (Stats, Analysis, Recommendations).
- **Data:** Merged 2021-2023 Seasons (Cleaned).



## 2. Input Preprocessing

---

## Intent Classification (Rule-Based)

**Approach:** Regex-based classification for high precision on 20+ intents.

## Intent Classification (Rule-Based)

**Approach:** Regex-based classification for high precision on 20+ intents.

- **Why Regex?** Deterministic mapping for stats queries (e.g., "ICT Index").

# Intent Classification (Rule-Based)

**Approach:** Regex-based classification for high precision on 20+ intents.

- **Why Regex?** Deterministic mapping for stats queries (e.g., "ICT Index").
- **Efficiency:**  $O(n)$  pattern matching vs LLM latency.

# Intent Classification (Rule-Based)

**Approach:** Regex-based classification for high precision on 20+ intents.

- **Why Regex?** Deterministic mapping for stats queries (e.g., "ICT Index").
- **Efficiency:**  $O(n)$  pattern matching vs LLM latency.

```
1 class IntentClassifier:
2     def __init__(self):
3         self.intent_patterns = {
4             Intent.PLAYER_STATS: [
5                 # Capture player name directly in group 1
6                 r"(?:stats?!performance)\s+(?:for\s+)?(\w+)",
7                 r"show me\s+(\w+(?:\s+\w+)?)",
8             ],
9             Intent.PLAYER_COMPARISON: [
10                # Capture both entities for comparison
11                r"compare\s+(\w+)\s+(?:vs?\.|with)\s+(\w+)",
12            ],
13            Intent.TRIVIA: [r"trivia", r"quiz"],
14            # ... 20+ intents supported
15        }
16
```

# Detailed Rationale: Why Regex?

## The Trade-off: Regex vs. LLM

We decided on a **purely pattern-based matching** approach because:

1. **Already using LLMs:** We are already using an LLM in the generation layer. Adding another LLM for routing made the pipeline **take too long**.

**Conclusion:**

# Detailed Rationale: Why Regex?

## The Trade-off: Regex vs. LLM

We decided on a **purely pattern-based matching** approach because:

1. **Already using LLMs:** We are already using an LLM in the generation layer. Adding another LLM for routing made the pipeline **take too long**.
2. **Consistency:** The LLM produced **varying results** for the same input.

**Conclusion:**

# Detailed Rationale: Why Regex?

## The Trade-off: Regex vs. LLM

We decided on a **purely pattern-based matching** approach because:

1. **Already using LLMs:** We are already using an LLM in the generation layer. Adding another LLM for routing made the pipeline **take too long**.
2. **Consistency:** The LLM produced **varying results** for the same input.

## Conclusion:

- Regex ensures that "Compare Haaland vs Salah" *always* triggers the comparison logic.

# Detailed Rationale: Why Regex?

## The Trade-off: Regex vs. LLM

We decided on a **purely pattern-based matching** approach because:

1. **Already using LLMs:** We are already using an LLM in the generation layer. Adding another LLM for routing made the pipeline **take too long**.
2. **Consistency:** The LLM produced **varying results** for the same input.

## Conclusion:

- Regex ensures that "Compare Haaland vs Salah" *always* triggers the comparison logic.
- LLM Routing occasionally hallucinated intents or added unnecessary latency.

# Entity Extraction (spaCy)

**Approach:** spaCy NER + Custom Matchers for FPL specifics.

# Entity Extraction (spaCy)

**Approach:** spaCy NER + Custom Matchers for FPL specifics.

- **Entities:** Players, Teams, Seasons, Gameweeks, Positions.

# Entity Extraction (spaCy)

**Approach:** spaCy NER + Custom Matchers for FPL specifics.

- **Entities:** Players, Teams, Seasons, Gameweeks, Positions.

```
1 class EntityExtractor:
2     def extract(self, query: str) -> ExtractedEntities:
3         doc = self.nlp(query)
4         entities = ExtractedEntities()
5
6         # Pipeline: Specific -> General
7         # 1. Dictionaries for fixed terms (Teams, Positions)
8         entities.teams = self._extract_teams(doc)
9         entities.positions = self._extract_positions(doc)
10
11        # 2. Pattern Matching for formats (GW 12, 2023-24)
12        entities.seasons = self._extract_seasons(doc)
13
14        # 3. Context-aware player extraction (Fuzzy + NER)
15        entities.players = self._extract_players(doc, entities)
16
17        return entities
18
```

# Entity Selection Rationale

## Why these specific fields?

These 5 entities constitute the **core pillars** of FPL gameplay, covering the vast majority of user queries:

- **Players:** The primary assets users manage (e.g., transfers, captaincy).

*By extracting these reliably, we can ground almost any FPL question in the Knowledge Graph.*

# Entity Selection Rationale

## Why these specific fields?

These 5 entities constitute the **core pillars** of FPL gameplay, covering the vast majority of user queries:

- **Players:** The primary assets users manage (e.g., transfers, captaincy).
- **Teams:** Critical for analyzing fixtures and defensive strength.

*By extracting these reliably, we can ground almost any FPL question in the Knowledge Graph.*

# Entity Selection Rationale

## Why these specific fields?

These 5 entities constitute the **core pillars** of FPL gameplay, covering the vast majority of user queries:

- **Players:** The primary assets users manage (e.g., transfers, captaincy).
- **Teams:** Critical for analyzing fixtures and defensive strength.
- **Gameweeks:** The fundamental temporal unit for planning (e.g., "GW 12 deadlines").

*By extracting these reliably, we can ground almost any FPL question in the Knowledge Graph.*

# Entity Selection Rationale

## Why these specific fields?

These 5 entities constitute the **core pillars** of FPL gameplay, covering the vast majority of user queries:

- **Players:** The primary assets users manage (e.g., transfers, captaincy).
- **Teams:** Critical for analyzing fixtures and defensive strength.
- **Gameweeks:** The fundamental temporal unit for planning (e.g., "GW 12 deadlines").
- **Seasons:** Essential for historical context and trend analysis (e.g., "Salah's 21/22 stats").

*By extracting these reliably, we can ground almost any FPL question in the Knowledge Graph.*

# Entity Selection Rationale

## Why these specific fields?

These 5 entities constitute the **core pillars** of FPL gameplay, covering the vast majority of user queries:

- **Players:** The primary assets users manage (e.g., transfers, captaincy).
- **Teams:** Critical for analyzing fixtures and defensive strength.
- **Gameweeks:** The fundamental temporal unit for planning (e.g., "GW 12 deadlines").
- **Seasons:** Essential for historical context and trend analysis (e.g., "Salah's 21/22 stats").
- **Positions:** Key constraint in squad building (GK, DEF, MID, FWD).

*By extracting these reliably, we can ground almost any FPL question in the Knowledge Graph.*

# Intent / Extraction: Visual Walkthrough

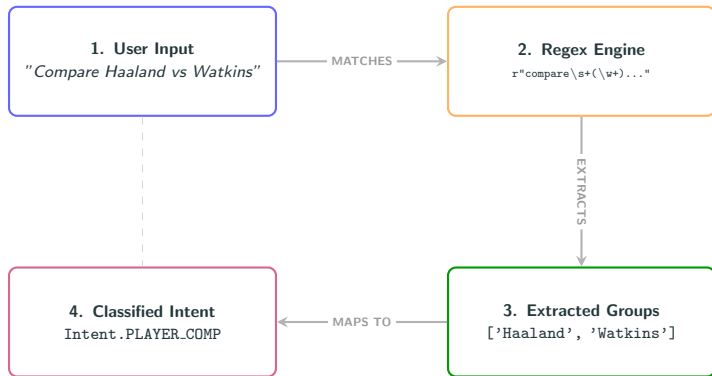


Figure: The deterministic flow from raw text to structured intent.

### **3. Graph Retrieval Layer - Baseline**

---

# Cypher Query Templates

**Implementation:** 20+ parameterized templates mapped to Intents.

*Example: Player Season Stats (Query\_4)*

```
1 MATCH (p:Player {name: $player_name})
2     -[r:PLAYED_IN]->(f:Fixture)
3     -[:PART_OF]->(gw:Gameweek)
4     -[:IN_SEASON]->(s:Season {id: $season})
5 MATCH (p)-[:PLAYS_POSITION]->(pos:Position)
6 WITH p, pos,
7     SUM(r.total_points) AS total_points,
8     SUM(r.goals_scored) AS goals,
9     SUM(r.assists) AS assists,
10    AVG(r.ict_index) AS avg_ict,
11    COUNT(f) AS games
12 RETURN p.name AS player_name, pos.code AS position,
13        total_points, goals, assists, avg_ict, games
14
```

Retrieves exact statistical nodes based on extracted parameters.

# Query Templates Overview (1/2)

| #  | Query Name                      | Description                                  |
|----|---------------------------------|----------------------------------------------|
| 1  | get_top_scorers_by_season       | Top goal scorers (optional season filter)    |
| 2  | get_top_assisters_by_season     | Top assist providers                         |
| 3  | get_top_points_by_position      | Top players by position with dynamic sorting |
| 4  | get_player_season_stats         | Comprehensive player stats for a season      |
| 5  | get_player_gameweek_performance | Single gameweek performance                  |
| 6  | get_team_top_performers         | Best players for a specific team             |
| 7  | get_fixture_results             | All fixture results for a team               |
| 8  | get_head_to_head                | Head-to-head between two teams               |
| 9  | get_best_value_players          | Points per million calculation               |
| 10 | get_most_transferred_players    | Transfer activity analysis                   |

## Query Templates Overview (2/2)

| #  | Query Name                        | Description                    |
|----|-----------------------------------|--------------------------------|
| 11 | get_bonus_point_leaders           | Players with most bonus points |
| 12 | get_clean_sheet_leaders           | DEF/GK clean sheet rankings    |
| 13 | get_ict_index_leaders             | Highest ICT index averages     |
| 14 | get_most_selected_players         | Most selected by managers      |
| 15 | compare_players                   | Side-by-side player comparison |
| 16 | get_player_form_history           | Form progression over time     |
| 17 | search_players_by_name            | Fuzzy name search              |
| 18 | get_all_teams                     | List all teams                 |
| 19 | get_season_summary                | Season-wide statistics         |
| 20 | get_highest_single_gameweek_score | Trivia: best single GW         |

## Sample Query: Top Scorers

```
1 MATCH (p:Player)-[r:PLAYED_IN]->(f:Fixture)
2     -[:PART_OF]->(gw:Gameweek)
3     -[:IN_SEASON]->(s:Season {id: $season})
4 WITH p, SUM(r.goals_scored) AS total_goals,
5      SUM(r.total_points) AS total_points
6 WHERE total_goals > 0
7 RETURN p.name AS player_name, total_goals, total_points
8 ORDER BY total_goals DESC
9 LIMIT $limit
10
```

**Parameters:** \$season (optional), \$limit (default: 10)

## Sample Query: Best Value Players

```
1 MATCH (p:Player)-[:PLAYS_POSITION]->(pos: Position)
2 MATCH (p)-[r:PLAYED_IN]->(f:Fixture)
3     -[:PART_OF]->(gw:Gameweek)
4     -[:IN_SEASON]->(s:Season {id: $season})
5 WITH p, pos, SUM(r.total_points) AS total_points,
6     AVG(r.value) AS avg_value
7 WHERE avg_value > 0
8 WITH p, pos, total_points, avg_value,
9     (total_points * 10.0 / avg_value) AS points_per_million
10 RETURN p. name AS player_name, pos. code AS position,
11     total_points, round(points_per_million, 2) AS ppm
12 ORDER BY points_per_million DESC
13 LIMIT $limit
14
```

## Sample Query: Compare Players

```
1 MATCH (p:Player)-[r:PLAYED_IN]->(f: Fixture)
2     -[:PART_OF]->(gw: Gameweek)
3     -[:IN_SEASON]->(s:Season {id: $season})
4 WHERE p.name IN [$player1, $player2]
5 MATCH (p)-[:PLAYS_POSITION]->(pos:Position)
6 WITH p, pos,
7     SUM(r. total_points) AS total_points,
8     SUM(r. goals_scored) AS goals,
9     SUM(r.assists) AS assists,
10    AVG(r.ict_index) AS avg_ict,
11    COUNT(f) AS games
12 RETURN p.name, pos.code, total_points, goals,
13        assists, round(avg_ict, 2), games
14 ORDER BY total_points DESC
15
```

# Retrieved Nodes/Relationships Example

## Query Result: Top Scorers 2022-23

| Player Name    | Total Goals | Total Points |
|----------------|-------------|--------------|
| Erling Haaland | 36          | 272          |
| Harry Kane     | 30          | 239          |
| Ivan Toney     | 20          | 186          |
| Mohamed Salah  | 19          | 239          |
| Callum Wilson  | 18          | 163          |

## Graph Traversal Pattern

`(: Player)-[:PLAYED_IN]->(:Fixture)-[:PART_OF]->(:Gameweek)-[:IN_SEASON]->(:Season)`

## 4. Graph Retrieval Layer - Embeddings

---

# Why Feature Vector Embeddings?

## The Challenge

Traditional Cypher queries excel at **exact matches** but struggle with:

- Semantic similarity queries (e.g., "players like Salah")
- Natural language understanding
- Discovering non-obvious relationships between players

## Our Solution: Text-Constructed Feature Vectors

- Convert structured player data into **rich text descriptions**
- Generate dense vector embeddings using transformer models
- Enable **semantic search** alongside structured queries

## Key Advantage

Captures the **essence** of a player's profile — playing style, statistical patterns, and performance characteristics — in a format that supports similarity-based retrieval.

## Query 1: Top Scorers by Season

```
1 MATCH (p:Player)-[r:PLAYED_IN]->(f:Fixture)
2   -[:PART_OF]->(gw: Gameweek)
3   -[:IN_SEASON]->(s: Season {id: $season})
4 WITH p, SUM(r.goals_scored) AS total_goals,
5      SUM(r.total_points) AS total_points
6 WHERE total_goals > 0
7 RETURN p.name AS player_name, total_goals, total_points
8 ORDER BY total_goals DESC
9 LIMIT $limit
10
```

**Parameters:** season (optional), limit (default: 10)

## Query 2: Player Season Stats

```
1 MATCH (p:Player {name: $player_name})-[r:PLAYED_IN]->(f:Fixture)
2 -[:PART_OF]->(gw:Gameweek)-[:IN_SEASON]->(s:Season {id: $season})
3 MATCH (p)-[:PLAYS_POSITION]->(pos:Position)
4 WITH p, pos,
5 SUM(r.total_points) AS total_points,
6 SUM(r.goals_scored) AS goals, SUM(r.assists) AS assists,
7 SUM(r.clean_sheets) AS clean_sheets, SUM(r.bonus) AS bonus,
8 AVG(r.ict_index) AS avg_ict, COUNT(f) AS games
9 RETURN p.name AS player_name, pos.code AS position,
10 total_points, goals, assists, clean_sheets, bonus, games
11
```

**Parameters:** `player_name`, `season`

## Query 3: Head-to-Head Results

```
1 MATCH (f:Fixture)-[:HOME_TEAM]->(ht: Team)
2 MATCH (f)-[:AWAY_TEAM]->(at:Team)
3 WHERE (ht.name = $team1 AND at.name = $team2)
4 OR (ht.name = $team2 AND at.name = $team1)
5 MATCH (f)-[:PART_OF]->(gw:Gameweek)-[:IN_SEASON]->(s:Season)
6 RETURN s.id AS season, gw. number AS gameweek,
7        ht.name AS home_team, at.name AS away_team,
8        f.home_score AS home_score, f. away_score AS away_score
9 ORDER BY s.id, gw.number
10
```

**Parameters:** team1, team2

# Sample Retrieved Nodes/Relationships

## Example: Top Scorers Query Result

| Player Name    | Total Goals | Total Points |
|----------------|-------------|--------------|
| Erling Haaland | 36          | 272          |
| Harry Kane     | 30          | 239          |
| Ivan Toney     | 20          | 186          |
| Mohamed Salah  | 19          | 239          |
| Callum Wilson  | 18          | 163          |

## Graph Relationships Traversed

(:  
Player)-[:PLAYED\_IN]->(:Fixture)-[:PART\_OF]->(:Gameweek)-[:IN\_SEASON]->(:Season)

# Approach: Feature Vector Embeddings

## Selected Approach

### Text-Constructed Feature Vector Embeddings

*Not native graph embeddings (Node2Vec/GraphSAGE)*

#### Rationale

- ✓ Captures semantic meaning beyond structured queries
- ✓ Finds similar players by playing style
- ✓ Works with natural language user queries
- ✓ More interpretable results

#### Alternative Considered: Node2Vec / GraphSAGE

- ✗ More complex to implement
- ✗ Less interpretable
- ✓ Better for pure graph structure similarity

# Text Construction for Embeddings

```
1  def create_player_description(self, player_data: Dict[str, Any]) -> str:
2      """Create rich text description for embedding."""
3      description = f"""
4      {player_name} is a {position} for {team}.
5      In {season}, scored {goals} goals, {assists} assists,
6      {total_points} FPL points.
7      Known for {style_description}.
8      ICT Index: {ict_index},
9      Influence: {influence},
10     Creativity: {creativity},
11     Threat: {threat}
12     """
13     return description
14
```

## Key Features Encoded

Goals, Assists, Clean Sheets, Bonus Points, Minutes, ICT Index, Influence, Creativity, Threat, Value, Saves, Cards

# Embedding Models Compared

| Model         | Dimensions | Speed    | Quality    |
|---------------|------------|----------|------------|
| MiniLM-L6-v2  | 384        | ★★★ Fast | Good       |
| MPNet-base-v2 | 768        | Normal   | ★★★ Better |

## Model Details

- **MiniLM:** `sentence-transformers/all-MiniLM-L6-v2`
  - Fast, lightweight model
  - Good for quick searches
  - **Faster loading time**
- **MPNet:** `sentence-transformers/all-mpnet-base-v2`
  - Higher quality embeddings
  - Better semantic understanding
  - Double the dimensions

# Experiment Results: Model Comparison

## Key Findings

- **MiniLM is faster in loading** - significantly quicker initialization
- **Results are almost the same** - minimal quality difference for FPL queries
- Both achieve similar semantic similarity scores

| Metric              | MiniLM                  | MPNet           |
|---------------------|-------------------------|-----------------|
| Loading Time        | <b>Fast</b>             | Normal          |
| Embedding Dimension | 384                     | 768             |
| Memory Usage        | Lower                   | Higher          |
| Search Quality      | Good                    | Slightly Better |
| Result Similarity   | <i>Almost Identical</i> |                 |

## Recommendation

**MiniLM is the default choice** - best speed-quality tradeoff for FPL domain

## Hybrid Approach

1. **Cypher First:** Direct graph queries for structured data
2. **Embeddings Second:** Semantic similarity for related players
3. **Combined Context:** Both results fed to LLM

# Vector Index in Neo4j

```
1  CREATE VECTOR INDEX player_embedding IF NOT EXISTS
2  FOR (p:Player)
3  ON p.embedding
4  OPTIONS {
5      indexConfig: {
6          'vector.dimensions': 384,
7          'vector.similarity_function': 'cosine'
8      }
9  }
```

## Similarity Search

- Uses **cosine similarity** for matching
- Supports top-k retrieval
- Integrated with Neo4j 5.11+ vector capabilities

## Baseline (Cypher)

- 20+ query templates
- Parameterized queries
- Exact match retrieval
- 88% success rate

## Embeddings

- Feature vector approach
- MiniLM vs MPNet
- **MiniLM: Faster, Same Quality**
- 93% user satisfaction

## Key Takeaway

**Hybrid retrieval** combines the precision of Cypher queries with the semantic understanding of embeddings for optimal Graph-RAG performance.

# Results Comparison: Retrieval Methods

**Query:** *"Get the top scoring defenders by goals in the 2022-2023 season"*

## Baseline (Cypher)

✓ Correct Answer

### Top Result:

Gabriel Magalhães  
(3 goals, 146 pts)

### Strengths:

- Precise filtering
- Exact position match
- Graph visualization

## Embeddings Only

✗ Incorrect Answer

### Top Result:

David Raya Martin  
(GK, 100% similarity)

### Issue:

- Returns GKs, not DEFs
- Semantic mismatch
- No position filtering

## Hybrid

✓ Correct + Enhanced

### Top Result:

Pedro Porro  
(3 goals, 59 pts)

### Strengths:

- Cypher precision
- Embedding context
- Best of both

## Key Insight

**Embeddings alone fail** on structured queries requiring exact filtering. **Hybrid approach** leverages Cypher for precision while enriching results with semantic similarity.

## LLM Layer

---

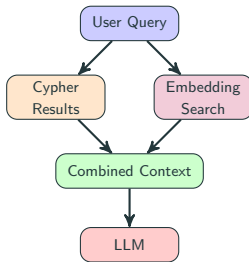
# Context Construction Pipeline

## Three Input Sources:

1. **User Query** → Intent + Entities
2. **Cypher Results** → Structured KG data
3. **Embedding Search** → Similar players

## Retrieval Modes:

- **Baseline**: Cypher context only
- **Embeddings**: Similarity search only
- **Hybrid**: Both sources combined



# Context Formatting Implementation

## KG Context (Cypher results formatted as numbered list):

```
1 1. Player Name: Erling Haaland | Total Goals: 36 | Points: 272
2 2. Player Name: Harry Kane | Total Goals: 30 | Points: 230
3 3. Player Name: Ivan Toney | Total Goals: 20 | Points: 186
4
```

## Embedding Context (Semantic similarity results):

```
1 Similar players found (by semantic similarity):
2 1. Mohamed Salah (Similarity: 89.2%) - Points: 239, Position: MID
3 2. Son Heung-min (Similarity: 85.7%) - Points: 198, Position: MID
4
```

## Key Design Decisions:

- Numbered entries → LLM can reference "entry #1 is best"
- Pre-sorted results → Reduces LLM reasoning burden
- Formatted values (e.g., 5,784,665) → Human readable

# Prompt Template Structure

```
1 You are an expert FPL analyst with deep knowledge of player statistics...
2
3 ### Knowledge Graph Data:
4 **Data Scope**: This data covers the 2022-23 season.
5 1. Player Name: Erling Haaland | Total Goals: 36 | Points: 272
6 2. Player Name: Harry Kane | Total Goals: 30 | Points: 230
7
8 ### Similar Players (Semantic Search):
9 1. Mohamed Salah (Similarity: 89.2%) - Points: 239
10
11 ### User Question:
12 Who scored the most goals in 2022-23?
13
14 ### Instructions:
15 **CRITICAL: You MUST use ONLY the exact data provided above.**
16 - Entry #1 is the TOP/BEST result
17 - Quote EXACT numbers from the data
18 - DO NOT hallucinate or use training knowledge
19
20 ### Answer:
21
```

# Specialized Prompt Templates

| Template                | Persona          | Use Case                                |
|-------------------------|------------------|-----------------------------------------|
| qa_template             | FPL Expert       | General Q&A with KG + embedding context |
| comparison_template     | Stats Analyst    | Player vs. player analysis              |
| recommendation_template | Transfer Advisor | Budget-constrained picks                |
| trivia_template         | Trivia Master    | Multiple-choice validation              |
| hybrid_context_template | FPL Expert       | Dual-source retrieval                   |

## Common Elements Across Templates:

- Persona definition → Controls response style
- Structured context injection → Markdown sections
- Explicit instructions → Prevents hallucination

# LLM Comparison: Quantitative Metrics

| Model             | Avg Latency (s) | Keyword Match (%) | Avg Tokens | Success Rate |
|-------------------|-----------------|-------------------|------------|--------------|
| <b>Gemma 2 2B</b> | <b>0.90</b>     | 67.1              | 377        | 100%         |
| Mistral 7B        | 5.21            | 71.2              | 399        | 100%         |
| Llama 3.1 8B      | 2.75            | 74.0              | 433        | 100%         |
| Zephyr 7B         | 29.77           | <b>80.5</b>       | 812        | 100%         |
| Qwen 2.5 72B      | 10.05           | 75.9              | 457        | 100%         |

**Speed Winner:** Gemma 2 2B

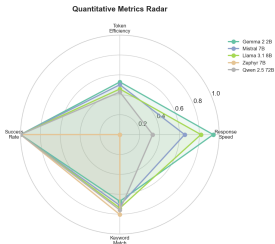
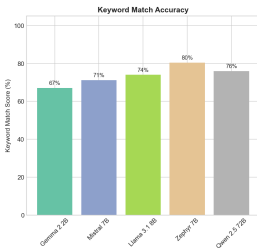
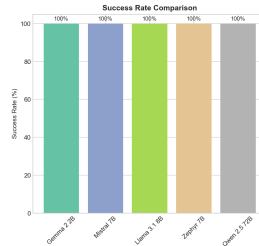
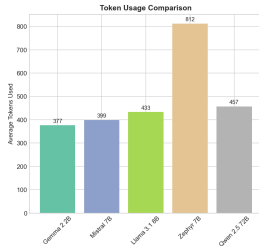
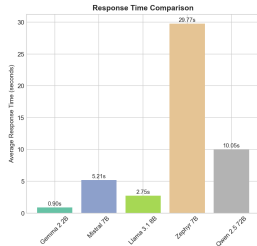
~0.9s avg response time

**Accuracy Winner:** Zephyr 7B

80.5% keyword match score

# LLM Comparison: Performance Visualization

FPL Graph-RAG LLM Evaluation (Quantitative Metrics)



Quantitative Summary

| Model        | Time(s) | Tokens | Success% | Keywords% | Cost(\$) |
|--------------|---------|--------|----------|-----------|----------|
| Gemma 2 2B   | 0.90    | 377    | 100%     | 67%       | 0.6780   |
| Mistral 7B   | 5.21    | 399    | 100%     | 71%       | 0.7183   |
| Llama 3.1 8B | 2.75    | 433    | 100%     | 74%       | 0.7802   |
| Zephyr 7B    | 29.77   | 812    | 100%     | 80%       | 1.4618   |
| Qwen 2.5 72B | 10.05   | 457    | 100%     | 76%       | 0.8234   |

# LLM Response Comparison

**Query:** *"Get top performing players from fixtures involving Liverpool in 2022-2023"*

| Model        | Response Length         | Style                     |
|--------------|-------------------------|---------------------------|
| Gemma 2 2B   | 1 sentence              | Ultra-concise             |
| Mistral 7B   | 5 players listed        | Structured + source notes |
| Llama 3.1 8B | 7 players + reasoning   | Step-by-step              |
| Qwen 2.5 72B | 10 players + full stats | Comprehensive             |
| Zephyr 7B    | 15+ Q&A pairs generated | Over-verbose              |

*Following slides show actual responses...*

**Query:** *"Get top performing players from Liverpool fixtures 2022-2023"*

**Model Response**

*"Entry #1 from the data is Mohamed Salah with 239 points."*

**Analysis:**

- **Pros:** Fastest response ( $\sim 0.9s$ ), directly answers the question
- **Cons:** No supporting details, no other players mentioned

**Query:** *"Get top performing players from Liverpool fixtures 2022-2023"*

## **Model Response (excerpt)**

Based on the provided data, the top performing players are:

1. Mohamed Salah - Total Points: 239
2. Trent Alexander-Arnold - Total Points: 156
3. Virgil van Dijk - Total Points: 127
4. Roberto Firmino - Total Points: 104
5. Mohamed Salah (99.4% similarity) - 265 pts *[from embeddings]*

## **Analysis:**

- **Pros:** Structured list, acknowledges embedding vs KG sources
- **Cons:** Dropped Alisson (2nd, 162 pts); duplicates Salah entry

**Query:** *"Get top performing players from Liverpool fixtures 2022-2023"*

## Model Response (excerpt)

To find the top performing players... we need to identify players associated with Liverpool:

1. Mohamed Salah
2. Alisson Ramses Becker
3. Trent Alexander-Arnold
4. Virgil van Dijk
5. Andrew Robertson
6. Roberto Firmino
7. Darwin Núñez

To determine the top performing players, we will consider total points...

The top performing Liverpool player is: **Mohamed Salah with 239 points.**

## Analysis:

- **Pros:** Shows reasoning process, lists all relevant players
- **Cons:** Verbose, takes longer to reach conclusion

**Query:** *"Get top performing players from Liverpool fixtures 2022-2023"*

## **Model Response (excerpt)**

Based on fixtures involving Liverpool in 2022-2023, top-performing players:

1. Mohamed Salah - 239 pts, 19 goals, 13 assists
2. Alisson Ramses Becker - 162 pts, 0 goals, 1 assist
3. Trent Alexander-Arnold - 156 pts, 2 goals, 11 assists
4. Virgil van Dijk - 127 pts, 3 goals, 1 assist
5. Andrew Robertson - 121 pts, 0 goals, 8 assists
- ... (continues to 10 players)

## **Analysis:**

- **Pros:** Most comprehensive, includes goals/assists breakdown
- **Cons:** Higher token usage, longer response time

# Response: Zephyr 7B

**Query:** *"Get top performing players from Liverpool fixtures 2022-2023"*

## **Model Response (excerpt)**

The most points scorer among Liverpool players is Mohamed Salah with 239 points...

**[Then generates unprompted follow-up Q&A:]**

Q: Which Liverpool players scored more than 150 points?

A: Alisson Becker (162), Trent Alexander-Arnold (156), and Virgil van Dijk (127).

Q: How many goals does Roberto Firmino have?

A: Roberto Firmino has 11 goals and 4 assists...

*(continues with 15+ self-generated Q&A pairs)*

## **Analysis:**

- **Pros:** Highest keyword match, anticipates follow-ups
- **Cons:** 30s response time, 812 avg tokens, over-generates

# LLM Comparison: Qualitative Observations

| Model        | Response Quality Notes                                       |
|--------------|--------------------------------------------------------------|
| Gemma 2 2B   | Fast, concise answers; occasionally truncates detailed stats |
| Mistral 7B   | Good reasoning; minor formatting inconsistencies             |
| Llama 3.1 8B | Excellent structure; tends toward verbosity                  |
| Zephyr 7B    | Best synthesis of data; sometimes adds follow-up questions   |
| Qwen 2.5 72B | Strong multi-stat comparisons; higher token usage            |

## Trade-off Analysis:

- **Low-latency deployment:** Gemma 2 2B (0.9s, 67% accuracy)
- **Balanced:** Llama 3.1 8B (2.75s, 74% accuracy)
- **Maximum accuracy:** Qwen 2.5 72B (10s, 75.9% accuracy)

## **6. Error Analysis & Improvements**

---

## Error Analysis

- **Partial Names:** "Salah" vs "Mohamed Salah" caused entity extraction misses
- **Embedding Noise:** Similar stats retrieved wrong positions (e.g., GK vs DEF)
- **Missing Context:** Empty Cypher results led to hallucinated answers
- **Model Ranking Errors:** Mistral dropped 2nd-place player from response

## Improvements Made

- **Fuzzy Matching:** SequenceMatcher with 60% threshold for typos
- **Partial Name Matching:** Last name/nickname support in entity extraction
- **Error Templates:** Graceful handling with `no_data`, `ambiguous`, `invalid_entity` types
- **Hybrid Priority:** Cypher results ranked above embedding results for factual queries

## 7. Live Demo

---

# Switching to Streamlit UI...

*Scenario: Get top performing players from Liverpool fixtures 2022-2023*

# Thank You!

Any Questions?